

McCore: A Holistic Management of High-Performance Heterogeneous Multicores

Jaewon Kwon
jaewon.kwon@yonsei.ac.kr
Yonsei University
Seoul, Republic of Korea

Yongju Lee
yongju.lee@yonsei.ac.kr
Yonsei University
Seoul, Republic of Korea

Hongju Kal
hongju.kal@yonsei.ac.kr
Yonsei University
Seoul, Republic of Korea

Minjae Kim
minjae.kim@yonsei.ac.kr
Yonsei University
Seoul, Republic of Korea

Youngsok Kim
youngsok@yonsei.ac.kr
Yonsei University
Seoul, Republic of Korea

Won Woo Ro
wro@yonsei.ac.kr
Yonsei University
Seoul, Republic of Korea

ABSTRACT

Heterogeneous multicore systems have emerged as a promising approach to scale performance in high-end desktops within limited power and die size constraints. Despite their advantages, these systems face three major challenges: memory bandwidth limitation, shared cache contention, and heterogeneity. Small cores in these systems tend to occupy a significant portion of shared LLC and memory bandwidth, despite their lower computational capabilities, leading to performance degradation of up to 18% in memory-intensive workloads. Therefore, it is crucial to address these challenges holistically, considering shared resources and core heterogeneity while managing shared cache and bandwidth.

To tackle these issues, we propose McCore, a comprehensive solution that reorganizes the heterogeneous multicore memory hierarchy and effectively leverages this structure through a hardware-based reinforcement learning (RL) scheduler. The McCore structure aims to enhance performance by partitioning the shared LLC based on each cluster's asymmetric computing power and conditionally enabling fine-grained access in small cores. The McCore RL agent holistically controls these structures, incorporating a hardware-based online RL scheduler that accounts for bandwidth utilization and caching effectiveness to consider the heterogeneity in McCore structures. By implementing the RL agent module as hardware that cooperates with existing hardware monitors and performance counters, low-latency scheduling is enabled without burdening the OS kernel. McCore achieves a 25.1% performance gain compared to the baseline and significantly outperforms existing state-of-the-art cache partitioning, sparse access managing schemes, and heterogeneous multicore schedulers, providing a comprehensive solution for high-performance heterogeneous multicore systems.

CCS CONCEPTS

• **Computer systems organization** → **Multicore architectures; Heterogeneous (hybrid) systems.**

KEYWORDS

multi-core architectures, heterogeneous computing, memory hierarchy, hardware-based scheduling, cache partitioning, reinforcement learning

ACM Reference Format:

Jaewon Kwon, Yongju Lee, Hongju Kal, Minjae Kim, Youngsok Kim, and Won Woo Ro. 2023. McCore: A Holistic Management of High-Performance Heterogeneous Multicores. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)*, October 28–November 01, 2023, Toronto, ON, Canada. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3613424.3614295>

1 INTRODUCTION

Heterogeneous multicore systems are becoming increasingly prevalent in high-performance computing due to various challenges [35, 69]. Instead of adding a few more high-performance large cores (L cores), incorporating many small cores (S cores) with high performance-power-area (PPA) efficiency leads to scaling of computational performance within limited area and power budgets.

However, we identify several challenges in adopting heterogeneous multicores for high-performance computing. First, using numerous S cores incurs two memory-related issues: 1) susceptibility to last-level cache (LLC) interference due to heterogeneous oblivious LLC management, and 2) increased memory bandwidth pressure with doubling core counts, leading to bandwidth waste. Additionally, intelligent scheduling is required to leverage heterogeneity and fulfill the needs of high-performance computing. This scheduler must continuously adapt to workloads and environments.

While prior work has proposed techniques to partially address these challenges with cache partitioning [34, 40, 78], sublining [37, 111, 112], and heterogeneous multicore scheduling [101, 102], no existing work holistically addresses all challenges. As demonstrated in [100], reorganizing memory hierarchy structure and designing structure-aware scheduling mechanisms are key to addressing asymmetric systems. To achieve holistic management of high-performance heterogeneous multicores, we must reorganize the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0329-4/23/10...\$15.00
<https://doi.org/10.1145/3613424.3614295>

memory hierarchy structure to alleviate LLC contention and bandwidth pressure, while designing an intelligent scheduler to make structure-aware decisions.

To this end, we present *McCore*¹, a system that restructures the memory hierarchy (*McCore Structure*) to mitigate LLC contention and memory bandwidth pressure, complemented by an RL scheduler (*McCore RL Agent*) designed to autonomously explore and exploit structural benefits.

McCore Structure presents an LLC management technique that partitions L and S cluster LLC and saves bandwidth waste by supporting conditional fine-grained memory access in S cores. The cluster-wise LLC is achieved through a novel cache indexing technique that leverages different hashes per cluster to point to exclusive LLC regions. The L cluster partition is much larger than the S cluster partition, considering asymmetric computational power. We demonstrate how we achieve a logically partitioned cache while preserving the benefits of inter-cluster shared LLC in data sharing and thread migration. We also present a utility-based balancing technique between the two partitions. In addition, *McCore Structure* proposes conditional fine-grained memory access in S cores, detailing the mechanism for supporting it using sub-ranked memory.

McCore RL Agent introduces a novel scheduling technique employing online RL for scheduling. The RL agent incorporates stats generated in *McCore Structure* into its state vectors to make structure-aware decisions. The RL agent continuously self-optimizes decisions using these states and adapts to workloads and *McCore* structures. The performance of RL agent is fundamentally driven by the volume of experiences from which it can learn. To promote effective RL learning through quantitative experience, we design a shared agent pooling experiences among cores. To further enhance the quality of gathered experiences, we employ optimization strategies that exploit spatio-temporal patterns in RL states (Section 5.3). Lastly, we co-design a hardware module for faster processing and reducing OS burden. Our hardware implementation only takes 0.34% of area and 0.02% of power compared to the baseline high-performance heterogeneous multicore [43].

We implement *McCore*, evaluating it using various workload mixes and comparing it to the baseline, state-of-the-art schedulers [102], cache partitioning [34], sublining schemes [37], and

hybrid of them [100]. Results (Section 7) show a 25.1% average performance improvement over the baseline (up to 48.3%), while also significantly outperforming competitors. We analyze *McCore*'s structure and RL agent, exploring their design space, and demonstrate the value of online learning even with a pretrained RL agent.

Overall, our contributions include:

- Tackling memory-related challenges in heterogeneous multicore systems, often overshadowed by PPA advantages.
- Introducing *McCore Structure*: a) Asymmetric computing power-aware LLC partitioning with a novel cache indexing technique to reduce small core interference. b) Decreasing bandwidth waste using conditional fine-grained access for small cores, enabled by sub-ranked memory.
- Proposing *McCore RL Agent*: An RL-based heterogeneous multicore scheduler for structure-aware decisions using online RL. a) Implementing algorithm and hardware co-design for effective learning. b) Showcasing the first RL-based heterogeneous multicore scheduling implementation since [36].

2 CHALLENGES IN HIGH-PERFORMANCE HETEROGENEOUS MULTICORES

In recent years, power constraints [35] and area limitations [69] have emerged as significant challenges in the design of high-performance computing chips. Consequently, heterogeneous multicore systems have become increasingly prevalent in high-performance computing environments to overcome these challenges. A prominent strategy in designing such systems involves the integration of numerous small cores instead of adding a limited number of extra large cores to the existing large cores. For example, Intel's 12900K [43] features eight large cores and eight small cores, with the latter occupying approximately 1/4 the area of the large cores [109]. This approach has the potential to deliver up to 50% more computational capabilities compared to the previous generations [41], while maintaining a similar power budget. Nonetheless, we identify memory-related challenges in heterogeneous multicore architectures that are often overshadowed by their PPA advantages.

2.1 Structural Challenges on Memory Hierarchy

Small Core's Interference on Shared LLC Figure 1 illustrates the impact of incorporating eight small cores in the eight large

¹Management of small (c) and large (C) core

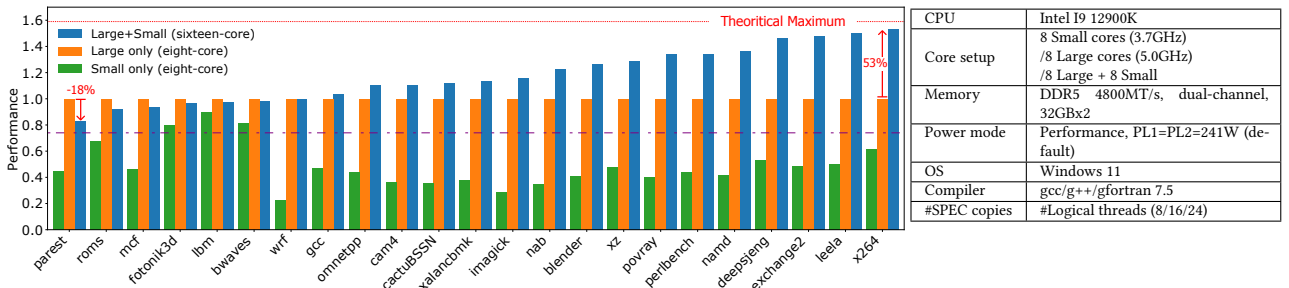


Figure 1: SPEC CPU2017 results for Intel 12900K: green, orange, and blue bars represent the performance of small cores, large cores, and their combined use, respectively. Normalized to the large-core-only case and sorted by scaling tendency (blue bars), the dotted line represents the frequency ratio between core types, with green bars above the line indicating higher IPC. All experiments were conducted on the real system. We used SPECrate metrics for performance.

cores. We compare this heterogeneous model with the homogeneous models that adopt eight small cores or large cores only. For some compute-intensive workloads (x264 and leela), the addition of numerous small cores results in performance gains approaching the theoretical maximum. However, for other workloads (gcc, omnetpp, and cam4), the benefits are limited to around 10% or less. Furthermore, integrating extra small cores can negatively affect memory-intensive workloads such as lbm, mcf, roms, and parest, leading to performance degradation of up to 18%.

This negative effect on the performance comes from the disparity of the dedicated computation and memory resources across cores. Large (L) and small (S) cores equally utilize memory resources on demand, although L cores outperform S cores in terms of computational throughput. Especially, in the case of memory-intensive workloads, S cores utilize a substantial portion of shared LLC resources, despite their limited computational capabilities. This leads to increased LLC misses because of capacity contention between L cores and S cores. Even worse, the additional computation provided by S cores rarely compensates for the performance loss in L cores. Therefore, the LLC management considering the heterogeneity of cores is essential to alleviate the interference of S cores on LLC.

Bandwidth Pressure with Doubling Core Counts The heterogeneity of cores causes not only the LLC capacity competition aforementioned but also the bandwidth competition of off-chip memory. This occurs because doubling the core count is not matched by the current DRAM bandwidth capabilities, resulting in excessive demand on memory bandwidth. In Figure 1, for fotonik3d, lbm, and bwaves, S cores alone (green) effectively saturate the main memory bandwidth, diminishing the benefits of L cores' high computational power (blue). In these situations, combining both core types results in doubling the demand on memory bandwidth, which outpaces the rate of memory bandwidth scaling.

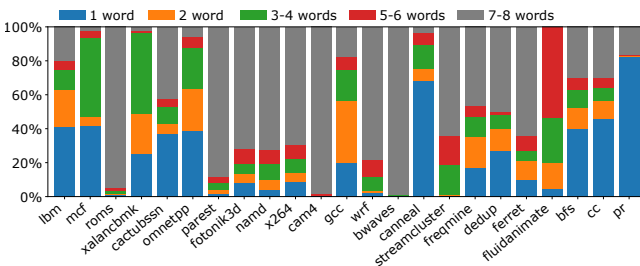


Figure 2: Distribution of words touched in a cache block in a baseline (§6) with 30MB, 12 way, 64-byte block LLC.

Additionally, a considerable number of these workloads cannot fully utilize the data received from DRAM, thereby consuming the DRAM bandwidth for unnecessary data. Figure 2 illustrates bandwidth utilization in the 12900K-like baseline, where the utilized words in 64B cache block size are counted until each block is evicted. It can be observed that many memory-intensive workloads utilize less than 50% of the available bandwidth. In the worst case (pr), the workload utilizes only 12.5% of the bandwidth for most of its duration. Given the already significant bandwidth demands of heterogeneous multicore architectures, it is crucial to address this issue of wasting off-chip bandwidth. In this paper, we aim to reduce

Table 1: Taxonomies of Prior works

Type	Structure		Sched.	Mechanism
	Shared LLC Contention	Limited Bandwidth	Heterogeneity	
Partitioning [34, 40, 78]	✓	✗	✗	Way Management
Sublining [37, 111, 112]	✗	✓	✗	Reduce BW Waste
PIE [102]	✗	✗	✓	Analytic Model
AMS [100]	✓	✗	✓	Part.-Driven Sched.
McCore (Ours)	✓	✓	✓	Struct. (§4) ← RL (§5)

bandwidth consumption for unnecessary data used in S cores, which alleviates the bandwidth contention for L cores.

2.2 Need for Intelligent Scheduling

A key challenge in high-performance heterogeneous multicore systems is the lack of intelligent schedulers that can continuously adapt to the situation and manage the heterogeneity of the processors. Since the primary purpose of S cores in these systems is to scale absolute performance, there is a need for more intelligent schedulers than previous works, which focused solely on energy savings in mobile systems through the use of S cores [20, 31, 55, 62, 63, 99]. However, state-of-the-art schedulers targeting this objective rely on analytic models [100, 102], limiting the full potential of heterogeneity.

One promising approach to achieving intelligent scheduling is through reinforcement learning (RL), an online model-free machine learning technique that self-optimizes decision-making without predefined knowledge or supervision [50, 94]. Nevertheless, a critical drawback of RL in scheduling is its propensity to exploit random chance until sufficient experience is accumulated. Moreover, implementing RL within the OS necessitates frequent kernel memory references, substantially affecting user-mode IPC performance [87]. Previous work [36] proposed the concept of adopting RL to self-optimize OS schedulers in heterogeneous multicores, but no implemented cases have been reported for over a decade due to these challenges. In this work, we present a novel approach to RL-based scheduling that leverages the massive core counts in high-performance heterogeneous multicore systems to overcome these obstacles.

3 BACKGROUND AND MCCORE

In high-performance heterogeneous multicore systems, addressing aforementioned challenges demands a holistic approach. The scheduler should be cognizant of shared resources and individual core heterogeneity, while shared cache and bandwidth management must also consider the heterogeneity of individual cores. Regrettably, no prior works take a holistic approach. Table 1 presents taxonomies of prior works addressing one or more challenges discussed.

- **Cache Partitioning** aims to address shared LLC contention problems. A prominent direction is way-based management across different workloads [34, 78], which has been applied to real products [40]. However, these methods are still oblivious to heterogeneity and do not resolve the disparities discussed in Section 2.1. Additionally, way-partitioning often impairs performance because it provides a severely limited number of partitions due to

skyrocketing core counts (16-64) and decreased LLC associativities (12-16) in recent processors [3, 4, 39, 43, 106, 108] compared to [25–30].

- **Sublining** aims to reduce bandwidth and energy waste of 64B coarse-granularity access by enabling more fine-grained access (sublined access). However, previous works [37, 111, 112] does not consider shared LLC contention and core heterogeneity.
- **Heterogeneous Multicore Scheduling** seeks to leverage asymmetric resources of heterogeneous multicores. Although prior work [102] does not consider limited bandwidth and shared LLC contention, the latest AMS [100] achieves scheduling and partitioning concurrently through way-partitioning driven scheduling. However, they do not address bandwidth limitations, and the aforementioned drawbacks of way-partitioning hinder their potential.

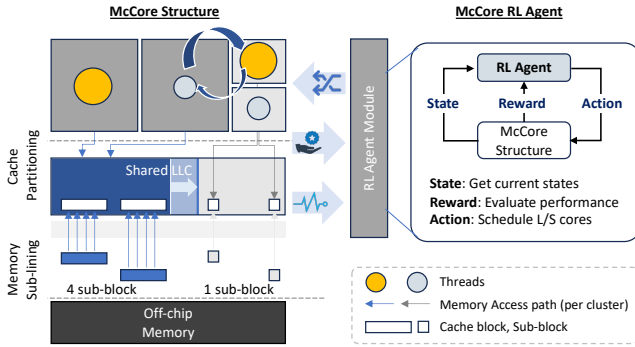


Figure 3: Overview of McCore

To address these challenges, we propose **McCore** (Figure 3), a novel approach that reorganizes the memory hierarchy of the heterogeneous multicore (*McCore structure*) and employs an RL-based scheduler (*McCore RL Agent*) to schedule workloads while considering the heterogeneity of this structure.

In the **McCore structure**, we employ shadow indexing to establish logical partitions within the shared LLC. This logical partitioning mitigates interference from S cores while preserving the advantages of shared LLC in terms of data sharing and coherence. Compute-intensive workloads or those with high data locality are allocated to L cores, leveraging their computational power and a larger portion of the LLC. Conversely, memory-intensive workloads with sparse data locality are confined to the smaller S partition. Here, fine-grained accesses adeptly manage frequent cache misses, saving memory bandwidth waste. Upon optimal workloads-structure mapping (scheduling), McCore structure harmoniously balances both computational and memory resource utilization, driving peak system performance.

The **McCore RL Agent** plays a pivotal role to ensure the McCore structure operates at its peak. Within the McCore Structure, the scheduling intricacies are framed as an RL problem. McCore RL Agent observes several workload and system-level features pertinent to the McCore Structure, treating these as a *state* from which it determines the appropriate scheduling *action*. Each thread receives a predefined *reward* relating to its performance within the McCore Structure, transitioning to a new *state* as a consequence of an action.

This sequence of state, action, and reward is called an *experience* in RL. The RL Agent learns from these experiences, refining its decision-making process for future iterations. To rapidly learn and ensure consistent decision-making across multiple threads, McCore RL Agent shares experiences among processors. We further detail the architectural support that enhances the efficiency of the McCore RL agent.

4 MCCORE STRUCTURE

In this section, we present how we reorganize the memory hierarchies of heterogeneous multicore.

4.1 Shadow Indexing

Cache locations, unlike memory locations, are not directly visible to software, as cache indexing is managed by hardware-implemented hash functions [18, 83]. The simplest form of hash function for cache indexing involves masking and modular arithmetic using set index bits. Hardware manufacturers choose specific hash functions [85, 105] to map memory addresses to cache locations. As software cannot directly access these locations, there is significant design space for managing cache using hash functions [79, 80]. Cache partitioning can be achieved by leveraging hash functions that point to exclusive regions (partitions) of addresses, allowing for more efficient cache management.

Cluster Partitioning via Index Hash Function: We present our approach for achieving logical partitioning of a physically shared LLC between L and S cores, while preserving associativities. The idea is to use two different hashes for each cluster that point to the exclusive region of each other. In Figure 4(a), each cluster has a primary hash with the other cluster's primary hash acting as a shadow hash. When accessing the LLC, both hashes are used in parallel: the primary hash points to the sets dedicated to each cluster (*primary location*), and the shadow hash points to the sets in the other cluster's region that do not belong to them (*shadow location*). An access to a certain address (0xd1ab916c0) always looks up the same pairs of locations pointed to by the two hashes regardless of the requestor, which are called *duo*. Within a duo, all cores can reliably index cache blocks without tracking specific sets, enabling flexible block placement and migration without informing block owners.

How it works?: With shadow indexing, newly inserted blocks in the LLC are exclusively inserted in the primary location pointed by the primary hash. This ensures that requestors can only use the partitioned region deployed to their respective clusters. This prevents compute-intensive workloads or workloads with high locality in L cluster from suffering interference by workloads isolated in S cores. Moreover, this logical partitioning does not compromise the precious associativity in each set, allowing for way-based cache management scheme to be orthogonally applied.

In addition, the benefit of shared cache in thread migration and data sharing is not compromised with the use of shadow locations. When inter-cluster thread migration or data sharing occurs, shadow locations are accessed. For example, suppose thread A is running on an L core and inserts a cache block corresponding to address 0xd1ab916c0 in the primary location. If thread A is moved to an

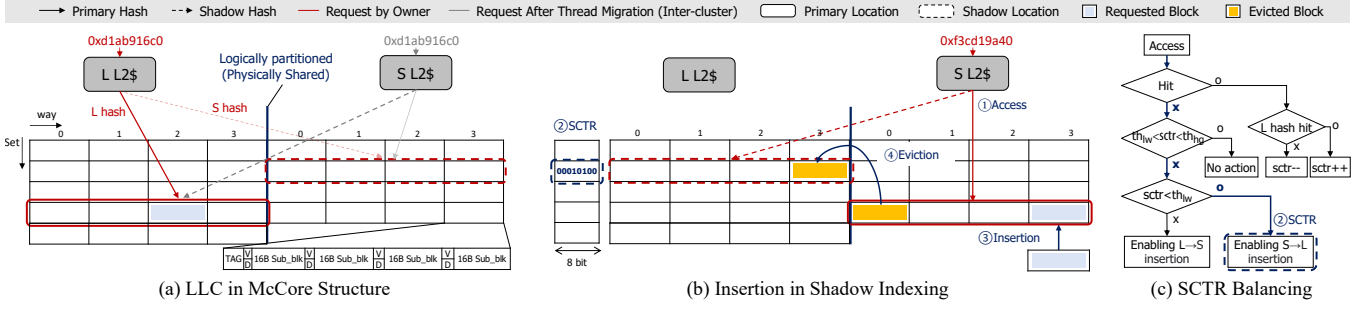


Figure 4: McCore LLC design

S core, it can still access this block by peeking the shadow location, thus preventing redundant copying of data in migration. This ensures that the benefits of shared cache are maintained.

Balancing the Partitions: McCore balances the utilization of sets in each duo by employing 8-bit saturating counters (SCTRs) per duo². Consider a scenario where x264 is running on the L core and mcf on the S core. Here, mcf heavily utilizes the LLC, while x264 mainly utilizes the L core’s computational power with minimal LLC usage. In this case, we balance the duo by allowing mcf to access shadow locations. Initially, SCTRs in each duo are set to 0b10000000, enforcing rigid partitioning. SCTRs increase on L partition hits and decrease on S partition hits. Over time, as mcf heavily utilizes the S partition and x264 seldom uses the L partition, SCTRs keep decreasing and fall below the threshold low (th_{lw}).

Balancing occurs during the insertion process, as illustrated in Figure 4(b). When an access to 0xf3cd19a40 misses both locations, the block is inserted into the primary location (①). The LLC controller checks the corresponding duo’s SCTR and notices it is below th_{lw} (②). Later, when a requested block is inserted into the primary location (③), the evicted block is placed in the shadow location (④). As mcf increasingly utilizes the shadow location in the L partition, the SCTR gradually increases. If the SCTR exceeds th_{hw} , the duo reverts to a partitioned state. This process balances the utilization of sets in each duo. Similarly, when the SCTR is higher than th_{hg} , workloads in the L core can utilize shadow locations in the S partition. The SCTR operation is summarized in Figure 4(c).

4.2 Fine Grained Access Support

The McCore structure addresses two objectives by supporting fine-grained memory access from S core: saving bandwidth waste and effectively handling sparse or irregular memory access, and preventing bandwidth contention from S core by throttling their memory access requests. To achieve these goals while also minimizing tag overhead and enabling inter-cluster thread migration, McCore’s LLC (and L1D/L2 caches of S core) is composed of a sector cache, with each 64B line divided into 16B sub-blocks with valid (V) and dirty (D) bits (Figure 4(a)). When L cluster inserts data into the cache, it brings in full sectors. On the other hand, S cluster insertion occurs at the sub-block level only when the bandwidth utilization

of the package or chip is high, ensuring efficient use of available resources.

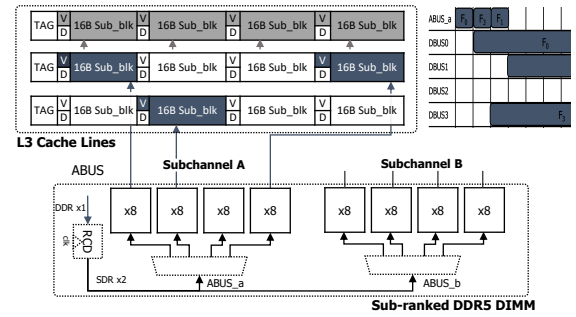


Figure 5: Examples of Sub-ranked DDR5 DIMM

In order to support fine-grained access without modifying the DRAM burst length, we adopt a sub-ranked DRAM design. Figure 5 shows an illustrative design of a sub-ranked DDR5 DIMM, where the 64-bit data bus (8 × 8) is divided into 8 sub-ranks. DDR5 already introduced sub-channels [65], where we can make use of 32-bit sub-channels independently so that we can fill two different cache lines in a single burst, maximizing parallelism. For fine-grained access, a register/muxed address triggers a single sub-ranked access at each cycle. One of the four ×8 sub-ranks in each sub-channel independently transfers the data to 16 times (DDR5 burst length), filling 16B of sub-blocks, while the address bus can trigger another fine-grained access each cycle. Furthermore, this can be done for two sub-channels in parallel. Thus, we can utilize a significant portion of the data bus with fine-grained support. Handling coarse-grained access, where we adopt [111], is split into multiple fine-grained accesses and prioritized accordingly.

4.3 Cost Analysis of McCore structure

We evaluated the McCore structure using CACTI 7.0 [9] with 22nm ITRS models. Table 2 summarizes the access latency, energy, and area of both set-associative cache and McCore with shadow indexing and sector cache designs. The cache size is 8 banked 32MB, and associativity is 8/16 way, which are the closest configurations to the 12900K LLC (30MB, 12-way, 10 bank). For shadow indexing, we measured the overhead of both shadow indexing and SCTR. We took a similar approach in measuring skewed associative cache overheads [79, 88].

²The number of duos is determined by the hash function type and the number of sets in each partition. In this paper, we assume modular hashing within each cluster partition and that the L partition size is a multiple of S partition size (e.g., 2:1, 3:1). Given these conditions, the number of duos corresponds to the number of sets in the L partition, enabling access to the SCTR of each duo using the L’s primary hash.

Table 2: Comparison of different cache types in terms of access latency, energy, and area

Cache Type	Access Latency (ns)	Energy (nJ)	Area (mm ²)
Normal SetAssoc 8-Way	4.286	0.759	34.87
Normal SetAssoc 16-Way	4.646	1.171	35.31
Normal SetAssoc 32-Way	5.123	1.662	36.21
Shadow Indexing 8-Way	4.286	1.072	34.91
Shadow Indexing 16-Way	4.646	1.522	35.33
Shadow Idx 8-Way Sector \$	4.485	1.086	35.46
Shadow Idx 16-Way Sector \$	4.845	1.536	35.86

Table 2 shows that shadow indexing mostly impacts the load energy (1.29 -1.41 $\times$) while the area and latency overheads are negligible (<0.1%). However, the power overheads of enabling shadow indexing are still cheaper than doubling the associativity (1.42 - 1.54 $\times$) where latency is also degraded (8.4 -10.2%). On the other hand, sector cache impacts the access latency (4.3 - 4.6%) due to extra V, D bit lookup, while energy overheads (0.9 -1.3%) and area overheads (1.5%) are small. We will consider these overheads when evaluating the McCore structure later.

Considerations for McCore Structure: While the McCore structure presents advantages, there are also several things to be carefully managed. First, to manage the coherence traffic and the burden on the snoop filter due to the fine-grained accesses of S cores in a non-inclusive LLC, the McCore structure leverages the natural snoop filter benefits of an inclusive LLC design [8]. The inclusive LLC's ability to determine the locations of valid subblocks, be it in the private cache, LLC, or DRAM, becomes instrumental: In the McCore design, maintaining coherence at 64B granularity implies that if an S core modifies the first subblock (termed α) in its cache, all other subblocks in the corresponding cache line, such as α and the fourth subblock (δ), are invalidated in the other S cores. The inclusive LLC, illustrated by the second cache line in Figure 5, comprised of α and δ , retains the unmodified δ along with the modified α . At this point, the V bit for α is unset and the D bit set, indicating α 's modification but its write-back to the LLC is still pending. When an L core requests a 64B line that includes α , the LLC downgrades the corresponding 64B in the S core from Modified to Shared state, retrieves the modified α from the S core's private cache, and sets the V bit for α . The LLC then fetches the second and third subblocks from DRAM, combines them with the already valid α and δ , and sends the complete 64B line.

Second, fine-grained access can overlook the high spatial locality of the instruction cache (I cache). Therefore, even when bandwidth utilization is high, the I cache of S cores is always accessed in a coarse-grained manner.

Third, shadow indexing may degrade energy efficiency, although a predictor can be used to serialize or parallelize duo access and reduce energy costs [77]. In this case, a separate SCTR for each cluster of each duo is necessary to determine access manners, and the merged value of two SCTR should be used to balance each duo.

Lastly, the effectiveness of the McCore structure heavily depends on an intelligent scheduler that is aware of McCore structure. The structure alone is workload-oblivious and assumes optimal task

Table 3: State (Feature), Action, and Reward

Type	Name	Description
State	#L1D miss	Memory intensity and cache effectiveness, 16-level for each
	#Mem req.	Compute intensity, 8-level (0-3: S core, 4-7: L core)
	Port util	Contemporary chip environment, 8-level
	BW_{mem} util.	
Action	ml	Must go to L core or wait.
	pl	Prefer L core with flexibility for scaling throughput.
	ms	Must be isolated to S core.
Reward	R_r	Reward of issued thread: #Instructions Retired/CPU Cycles
	R_p	Penalty for incompetent decision: 0.01

allocations within each cluster—compute-intensive tasks on L cores and memory-intensive tasks on S cores. Additionally, recognizing and efficiently scheduling workloads with irregular memory access on S cores is crucial. In the subsequent section, we delve into the design of an RL-based scheduler tailored to these requirements.

5 MCCORE RL AGENT

In this section, we present the design of the McCore RL agent, along with its optimization techniques and hardware support.

5.1 State, Action, and Reward

Before discussing the detailed mechanism of RL-based scheduling in McCore, we briefly examine the state, action, and reward employed in the system, as shown in Table 3.

State: The state used in McCore consists of four different features : #L1D misses, #Memory requests, Port utilization, and BW_{mem} utilization. The first two features (per time) are related to caching effectiveness [73] and memory intensiveness [110], while the latter two are related to compute intensiveness and contemporary chip environment. The RL agent learns from these features to enhance the synergy between the McCore structure and the prevailing workloads.

Action: McCore supports three different actions: ml (must go to the L core or wait), pl (prefer L core with flexibility for scaling throughput), and ms (must be isolated to the S core). The threads with same action go to same decision queue in RL agent module. The scheduling priority among actions are $ml = ms > pl$.

Reward: The reward is defined as the #Instructions Retired per CPU Cycle (IPC) of the issued thread. However, if the agent's decision results in the thread not being selected for scheduling, it will be penalized with a penalty of R_p .

5.2 Scheduling

As shown in Figure 6(a), the McCore RL scheduler comprises three main hardware components: Scheduler Queue (SQ), Q-value Table (QV_table), and Evaluation Queue (EQ). QV_table stores expected Q-values for every state-action pair, integrating feedback from all 16 cores into a single table for sample density and shared thread experiences. The EQ acts as a FIFO queue, preserving the update order of issued threads and waiting for thread stats within the scheduling quantum. The scheduler employs an auxiliary SQ to dynamically determine optimal actions for each thread during scheduling, rather than predefining actions in updates, thus improving adaptability and responsiveness.

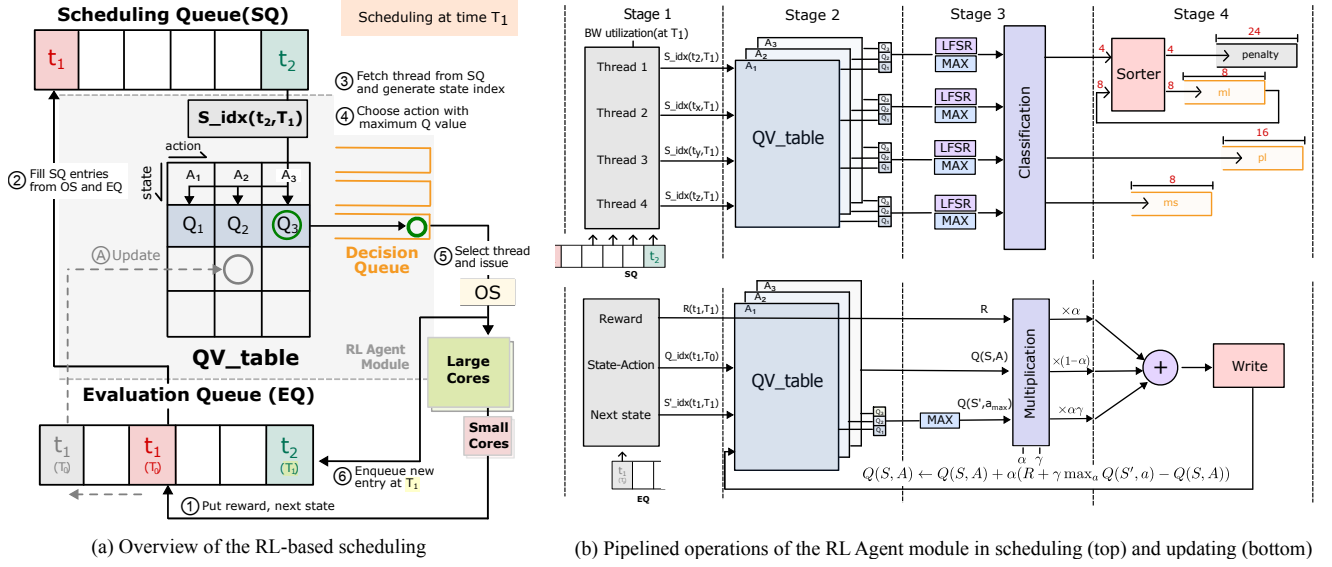


Figure 6: McCore RL-based thread scheduling overview

During each scheduling interrupt, the initial step ① in McCore's RL-based scheduling process is enqueueing each thread's next state (S') and reward (R) into the corresponding EQ entry, using a key generated from the thread's tid (t_1) and issue time (T_0). This maintains the order of issued threads, which is crucial for later update (A). Concurrently, ② the operating system populates the scheduler queue (SQ) with waiting threads. If SQ is not full, the McCore agent adds the current thread. The SQ filling policy is determined by the operating system, such as the CFS [74] policy. ③ The McCore agent generates the state index (S_idx) for each thread in the SQ, considering the thread's last record (S') and current resource utilization. ④ Then, the agent searches the thread's state index in the QV_table and finds the action A_t with the highest Q-values. ⑤ After determining the optimal action for each thread, the agent selects a thread and suggests the corresponding action. Selection occurs in two stages: first, ordering threads based on the decision queue ($ml = ms > pl$), and then within each queue (ml : Q-value order, pl, ms : SQ order). The operating system typically follows the McCore agent's suggestion and issues the selected thread to an available core. Simultaneously, ⑥ the state-action pair of t_2 is enqueued in the EQ entry using a key derived from t_2 's tid and issue time. Steps ①–⑥ repeat for each scheduling interrupt, enabling continuous learning and improvement through dedicated hardware modules.

(A) The McCore agent module updates the QV_table in the order maintained by the EQ entries when there is no scheduling interrupt. The agent uses the Bellman equation, a recursive formula calculating the expected Q-value for each state-action pair based on obtained rewards and the maximum Q-value of the next state. Updates are decoupled from the scheduling process and use dedicated hardware, ensuring no impact on the scheduler's critical path. Optimization techniques for the update path are discussed later in the section.

Hardware Design of RL Agent Modules: The hardware modules for the RL agent were co-designed to effectively process the scheduling and updating tasks. The RL agent module consists of a 4-stage pipelined structure as shown in Figure 6(b), enabling the concurrent processing of four threads during scheduling. Stages 3 and 4 are the most critical, corresponding to ④ and ⑤ in Figure 6(a). In stage 3, the agent generates random numbers using linear feedback shift registers (LFSR) to balance random exploration and exploitation. With a probability of ϵ , the agent explores random actions; otherwise, it exploits the action with the highest Q-values. Threads are then classified based on their chosen action. In stage 4, threads are sorted and placed into their respective decision queues as described in Section 5.1. Threads choosing ml are sorted by Q-values for optimal throughput. The 8-entry ml queue accommodates the top eight threads among four new and eight existing entries, with other threads directed to the penalty queue and receiving R_p . The agent maintains FIFO order within ms and pl queues to ensure fairness. Threads are selected based on action priorities and respective queue orders. If there are any remaining unscheduled S cores after this selection, to avoid wasting idle resources, penalized threads from the penalty queue are assigned to fill these unoccupied S cores.

The same agent module behaves differently during updating, as illustrated at the bottom of Figure 6(b) via its 4-stage pipeline. Updates (A) occur in issue order to maintain progressive updates, as guaranteed by the Bellman optimality equation [13]. In the first two stages, the agent fetches the reward R , Q-values of the next state $Q(S', a)$, and the state-action pair $Q(S, A)$ of the last EQ entry with non-zero rewards. In stage 3, the agent selects the maximum Q-value among the outputs from port 1, denoted as $\max_a Q(S', a)$. The agent then multiplies three partial values R , $Q(S, A)$, and $\max_a Q(S', a)$ using predefined hyperparameters α and γ [104]. In stage 4, the agent sums these partial values and writes the updated $Q(S, A)$ value via port 0, the sole read/write port in the QV_table.

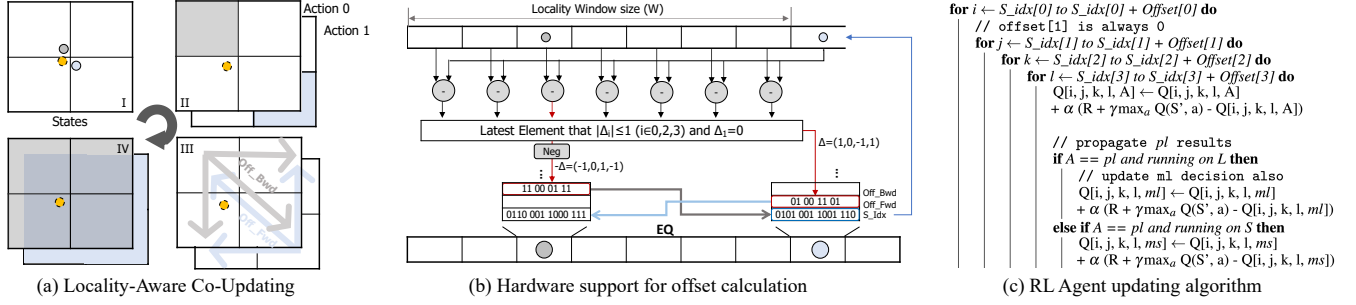


Figure 7: Locality-Aware Co-Updates in McCore's RL Agent

5.3 Exploiting Localities in the Update Phase

In McCore's decoupled update phase, we utilize several optimizations that expedite learning, circumventing the need for random exploration of unvisited states. The key idea is to exploit the spatio-temporal localities in RL states.

Locality-Aware Co-Updates: Spatio-temporal localities in RL states arise when a single thread exhibits similar phases over multiple scheduling quanta or when parallelized workloads have threads with similar behaviors. As shown in Figure 7(a)-I, these states might be adjacent, due to variations. Suppose grey dots represent the first visited states, followed by blue and then yellow dots. However, naive RL implementation, depicted in Figure 7(a)-II, does not exploit experiences of other threads or previous phases with high spatial locality, such as the grey and blue dots. The later visited yellow dots cannot exploit former similar behaviors and try their random luck. The insufficiency of experience forces the yellow dot, being the most recently visited and unexplored state, to rely on random exploration. This could have been avoided by piecing together the scattered experiences from the adjacent grey and blue states. McCore introduces locality-aware co-updates (Figure 7(a)-III), where threads calculate offsets between their state indices to represent *relative* pointers: The first visited grey state's *backward* offset (Off_Bwd) points to the *later* visited blue state's index; the blue state's *forward* offset (Off_Fwd) points to the *earlier* grey state's index. By using marked offsets in EQ entries, adjacent states are co-updated, exploiting spatio-temporal localities within a given temporal window and avoiding resolution degradation. This approach (Figure 7(a)-IV) enables efficient learning by leveraging experiences of adjacent states, such as the grey and blue dots, allowing threads visiting the yellow state to avoid random luck even during their first visits. This takes advantage of the spatio-temporal localities within RL states.

Hardware Support for Offset Calculation: McCore includes hardware support for calculating and storing offsets (Off_Fwd, Off_Bwd), as in Figure 7(a)-III in EQ entries. Offset calculators (Figure 7(b)) determine offsets for specific EQ entries (e.g., blue dot) by populating their S_idx (4-dimension state, 0101 001 1001 110) into queues storing the W most recently visited S_idx s (grey, 0110 001 1000 111). Each entry calculates the delta (Δ) between their S_idx and the populated S_idx . The calculator selects the latest entry that visited an adjacent state with a 1-difference in fine-grained features ($i=0,2,3$) and an exact match in the coarse-grained feature ($i=1$). The calculated Δ (four signed 2-bit values, 01 00 11 01) is placed into the Off_Fwd field in the EQ entry that populates the

S_idx , with Δ_1 always set to zero, serving as a valid bit. The sum of the blue S_idx and Off_Fwd equals the grey S_idx . Simultaneously, the calculator places the $-\Delta$ into the Off_Bwd field in the grey EQ entry, pointing to the blue S_idx .

As illustrated in Figure 7(c), the agent performs co-updates of states between S_idx and $S_idx + \text{Offset}$ during the update process, also propagating results of pl actions. Threads in the pl queue are scheduled later than those with confident actions (ml , ms). If a pl thread is issued to large cores, it implies the ml queue wasn't full, and the thread could have been in the ml queue. McCore co-updates $Q[S_idx, ml]$ and $Q[S_idx, pl]$, applying this to the opposite case as well. With these optimizations, a single EQ entry updates up to 32 QV_table entries. Processing these updates serially in the OS would consume significant CPU and memory resources. Instead, McCore RL agent modules (Figure 6(b)) process updates independently when no scheduling interrupts occur, ensuring a more efficient implementation.

6 EXPERIMENTAL METHODOLOGY

Modeled System: We use Sniper [17]—a widely adopted pin [44] based x86 multi-core simulator in the literature to evaluate heterogeneous multicore architectures. We evaluate the performance of McCore based on an Intel Alderlake [45, 46]-like baseline system which has heterogeneous 16 cores. Table 4 summarizes the configuration for our baseline system. The large cores and small cores are modeled on the Intel Goldencove and Gracemont³ microarchitectures respectively [5, 6, 21, 21–23, 46, 46]. The latency in memory hierarchy is measured by several microbenchmarks [48, 49, 64]. We measure the energy using McPAT [58] integrated in Sniper. When we simulate McCore, additional structure overhead discussed in Section 4.3 is considered. The partition size ratio between the L and S clusters is set at 3:1 with the SCTR threshold values of 32 for the low and 224 for the high. Fine grained access using sub-ranked DRAM is implemented similar to [112].

Schedulers: We implement the McCore RL agent with $\epsilon = 0.1$, $\alpha = 0.07$, $\gamma = 0.66$ means exploration rate, learning rate and discount rate, respectively [94]. We used a 1 ms time slice like prior works [101, 102] to exploit time-varying workloads behavior while keeping migration overhead small. In doing so, we modeled of

³The IQ, L1 I-cache, and decoder of small cores are modeled after Skylake [107], which Intel has reported to exhibit similar IPC as Gracemont [41]. Gracemont's unique frontend and instruction scheduling for power efficiency [22] differ from traditional Intel microarchitectures established in the simulator.

Table 4: Baseline System Configurations

Cores	Large core (L): 8 cores, x86-64, 8-wide issue, OoO, 5.0GHz, 512-entry ROB, 192-entry LQ, 114-entry SQ, 205-entry IQ Small core (S): 8 cores, x86-64, 5-wide issue, OoO, 3.7GHz, 256-entry ROB, 80-entry LQ, 50-entry SQ, 97-entry IQ
L1 I/D caches	L: 32KB/48KB, 8-way/12-way, 3/5-cycle, 16-entry MSHR S: 32KB, 8-way, 3-cycle, 10-entry MSHR
L2 caches	L: 1.25MB, private, 10-way, 15-cycle, 48-entry MSHR S: 2MB, shared by 4 cores, 16-way, 18-cycle, 64-entry MSHR
LLC	30MB, shared by 16 cores, 12-way, inclusive, 67-cycle (in large core cycle), 192-entry MSHR, LRU
Coherence	64B cache line, MESI protocol
Memory	DDR5 DRAM, 76.8GB/s, 49 ns access latency, memory controller queuing latency modeled

migration overheads [101] and find that it is less than 1% for a 1 ms time slice in McCore structure. The RL agent incur extra overheads. If the McCore agent only works as software, additional overheads due to long serial updates need to be considered. We measured using the captured record on simulation time in real machine and the overheads is up to 30 us, which is about 6 times of linux context switching latency [14]. In contrast, the McCore RL hardware module incur overheads less than 0.2 us. Moreover, it process update concurrently while thread is running on cores, does not affect the critical path of scheduling. We implement the RL agent using Chisel [24] with a storage configuration as in Table 5. We measure the power and area of this module at 100MHz frequency using Synopsys Design Compiler [95] with FreePDK 45nm library [71]. The result is scaled down to a 7nm situation.

Comparators: We compare the RL agent with state-of-the-art AMS [100] and PIE [102] schedulers, using random scheduling as the baseline. For the AMS scheduler, we use the cost difference between L and S cores for scheduling, implementing the AMS-greedy algorithm. Moreover, to examine AMS performance in high associativity LLC, we test it with double associativity. For the PIE scheduler, we found the memory-level parallelism (MLP) ratio estimation in [102] underestimated Skylake-level S cores [41], causing PIE to significantly underperform compared to the random scheduler. Instead, we benchmarked the MLP ratio on a real machine [57] and applied it to PIE. This modified version of PIE is deployed to comparators unless stated otherwise.

We compare McCore to state-of-the-art cache partitioning [34] and sublining [37] scheme. KPart [34] is implemented with their online profiling (DynaWay). The profiling quantum is adjusted to our environments. ISLE [37] is implemented for sublining. However, we adopt sub-ranked DRAM with partitioned command and data bus unlike ISLE’s flexible use of buses.

Workloads: We simulate both homogeneous and mixes of SPEC CPU2017 apps [89], and multithreaded apps from PARSEC [76] and GAPBS [10]. We picked 14 SPEC CPU2017 [89] workloads mostly in the left half of Figure 1 except x264 and namd. For multiprogrammed workloads, we generate 150M instruction traces using the Simpoint [84] tools. When measuring processing throughput, workloads are replayed until every trace is ended at least one time.

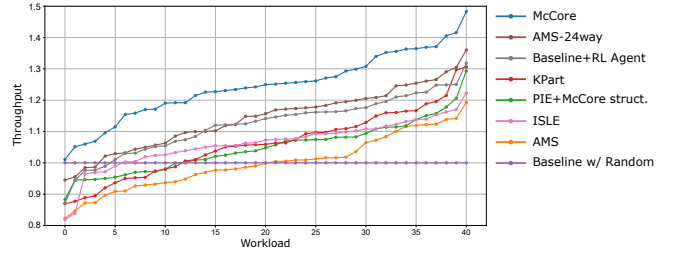
Table 5: Storage Configuration of the McCore RL Agent

Structure	Description	Size
QV_table	• 3 actions • 1 partial QV table for each action • 1 RD/WR port and 3 RD ports per partial table • #Entries in each partial table = feature dimensions (16 x 8 x 16 x 8) • Entry size = Q-value width (16b)	98KB
EQ	• #Entries = 128, W = 8 • Entry size = state (14b) + action (2b) + reward (9b) + next state (14b) + key (7b) + Off_Fwd (8b) + Off_Bwd (8b) + valid (1b)	1.2KB
SQ	• #Entries = 32 • Entry size = tid (20b) + last state index (11bit) + valid (1b)	0.1KB
Total		99.3KB

For multithreaded workloads, 1/runtime is measured instead of throughputs.

7 EVALUATION RESULTS

7.1 End-to-End Performance Comparison

**Figure 8: End-to-end performance comparison between McCore and other state-of-the-art approaches.**

We first conduct an end-to-end performance comparison of various schedulers and structures. Figure 8 displays the performance distribution across 41 mixes of randomly chosen 16 applications from the SPEC CPU2017. Each line indicates the performance gain compared to the random baseline. Both Kpart and ISLE are evaluated with the PIE scheduler.

The results reveal that AMS underperforms due to its low associativity, with about half of the workloads underperforming compared to the random baseline. Kpart slightly overcomes the way limitation through clustering, leading to better performance than AMS. ISLE saves significant bandwidth, resulting in substantial performance gains. However, contention issues limit its maximum gain compared to Kpart. When given sufficient ways (24-way), AMS performs well due to its tightly integrated scheduler and partitioning. The comparison between AMS and AMS-24way highlights the limitations of way-based partitioning in our environment.

McCore, with its comprehensive management capabilities, achieves an average performance gain of 25.1% (up to 48.3%). McCore outperforms other approaches, even surpassing AMS with 24-ways. The performance gains of McCore result from multiple factors. Comparing the baseline with the RL agent and the united McCore reveals the structural benefits of McCore. By comparing the McCore structure with PIE and the united McCore, we can observe the potential of the McCore RL agent. The synergistic effect of the structure and scheduler drives McCore’s performance improvements.

7.2 Analysis of McCore Structure

In this subsection, we evaluate homogeneous workloads to specifically highlight the structural benefits.

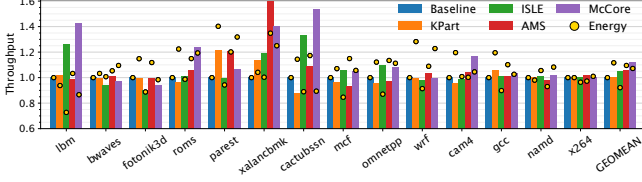


Figure 9: Structural comparison among Baseline, ISLE, KPart, AMS, and McCore.

Performance and Energy Impact of the McCore Structure:

Figure 9 presents the throughput of each scheme. We first discuss the effects of partitioning and sublining using McCore’s intelligent scheduling. McCore outperforms other schemes, achieving a 12.3% increase in geomean compared to the baseline. In memory-intensive workloads with significant bandwidth waste, such as lbm and cactubssn, McCore surpasses ISLE through cluster partitioning. The key factor is guaranteeing the reuse interval for the L cluster, which highlights that one reason for bandwidth waste is the inability to guarantee the reuse interval. Next, we examine the impact of mitigating LLC contention. In LLC-intensive workloads (xalancbmk, cam4, roms), AMS performs well due to its heterogeneity-aware partitioning. However, KPart rarely mitigates LLC contention in homogeneous workloads, as it employs heterogeneity-oblivious cache partitioning. Sublining also impacts several workloads. The lbm, cactubssn, and mcf experience performance improvements by increasing effective bandwidth in ISLE. However, this approach has a downside for bwaves and fotonik3d, where bandwidth waste is minimal. Lastly, we address the energy consumption trade-off of McCore. Shadow indexing increases the energy consumption of LLC-intensive workloads by up to 31% due to additional LLC indexing. Nevertheless, energy savings from sublining can compensate for the system’s energy consumption. On average, McCore consumes 7.2% more energy compared to the baseline.

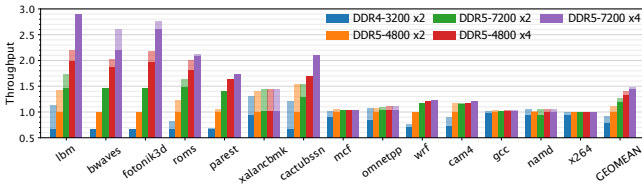


Figure 10: Bandwidth Sensitivities for McCore and Baseline.

Bandwidth Sensitivity: We investigate the bandwidth sensitivity of McCore and the baseline by adjusting the bandwidth from the default (DDR5-4800, 2 channels) through scaling channels and transfer speed. Figure 10 illustrates the bandwidth sensitivity, with the transparent portion of each graph indicating McCore’s performance gain over the baseline at the same bandwidth. Performance is normalized to the baseline at the default bandwidth.

Our analysis reveals that high-performance heterogeneous multicores are bandwidth-intensive. Workloads like lbm, bwaves, and fotonik3d see performance boosts of up to 2.9× when bandwidth

triples (DDR5-7200 x4). Other workloads (roms, parest, xalancbmk, and wrf) exhibit exponential growth up to 2× bandwidth. McCore’s sublining proves effective in bandwidth-limited scenarios, achieving a geomean performance gain of 1.47× for 3× bandwidth. For lbm, McCore’s performance at the default bandwidth approaches the baseline’s 1.5× bandwidth. Sublining benefits wane as bandwidth pressure lessens. Notably, McCore’s advantage in the highly LLC-intensive workload xalancbmk remains constant across bandwidth scaling, signifying cache partitioning’s impact. In summary, the McCore structure at the default bandwidth (1.12×) attains 2/3 of the effect of a 50% bandwidth increase (1.18×), attributable to the synergy of partitioning and sublining. This result underscores the McCore structure’s effectiveness in our environment.

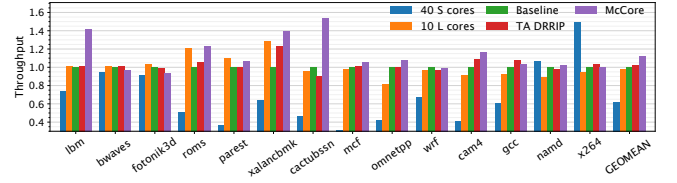


Figure 11: Other Sensitivity Analysis for McCore and Baseline.

Other Sensitivity: Figure 11 presents additional sensitivity studies for McCore and baseline. TA-DRRIP [51] slightly improves the baseline with LRU by 2.7% on average, yet shows limited impact on workloads like xalancbmk. The baseline’s strength in compute-intensive workloads is balanced by a weakness in memory-intensive tasks, yielding a meager overall advantage of 2.2% over 10 L. The interference effect in the 40 S configuration significantly degrades performance, resulting in a 38% decrease from the baseline.

Table 6: Multithreaded workloads and inputs used

Suite	Benchmark and input
PARSEC	canneal, streamcluster, freqmine, dedup, ferret, fluidanimate (all Simlarge)
GAPBS	bfs, cc, pr (all RMAT-24[19])

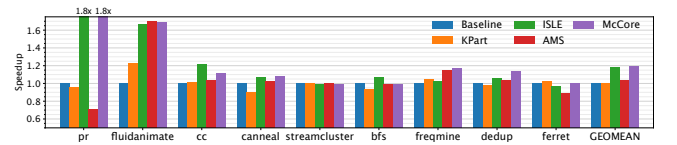


Figure 12: Multithreaded workloads.

McCore in Multithreaded Workloads: We evaluate McCore on several multithreaded workloads to assess whether the RL agent can effectively manage the McCore structure in such workloads. Each workload in Table 6 runs with 16 threads. Figure 12 presents the speedup of KPart, ISLE, AMS, and McCore. KPart exhibits limited gains due to its heterogeneity-oblivious partitioning. Although AMS performs well on fluidanimate and freqmine, the LLC in AMS is not shared among clusters, which is detrimental to workloads with frequent inter-cluster data sharing (pr and ferret). ISLE utilizes fine-grained access and achieves speedups of up to 1.8× for memory-intensive workloads with low locality (pr, cc, and fluidanimate). McCore leverages intelligent scheduling and partitioning for other applications, especially dedup and freqmine,

achieving additional speedups compared to ISLE with PIE. On average, McCore attains a 19% speedup relative to the baseline with a random scheduler.

7.3 McCore RL Agent Analysis

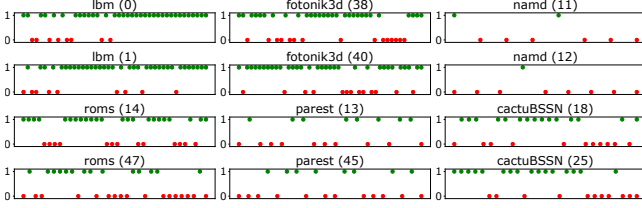


Figure 13: Decision history on a complex mix; green dots indicate threads on small cores.

Case Study for Decision Analysis: To gain deeper insight into the McCore RL agent, we run a long and complex mix of 50 traces to simulate real-world situations where the number of processes can exceed the core count. Figure 13 illustrates the scheduling history of some traces. The RL agent schedules compute-intensive workloads (e.g., namd) on L cores and heavily bandwidth-intensive workloads (lbm and fotonik3d) on S cores. Other workloads (roms, parest, and cactuBSSN) switch between large and small cores to adapt to the application phase. Considering that the PIE scheduler is already sufficient to handle consistent workload phases for namd, lbm, and fotonik3d, the performance gain of the RL agent comes from its ability to adapt to applications with dynamic behavior, reflecting a more comprehensive understanding of the application and the McCore structure.

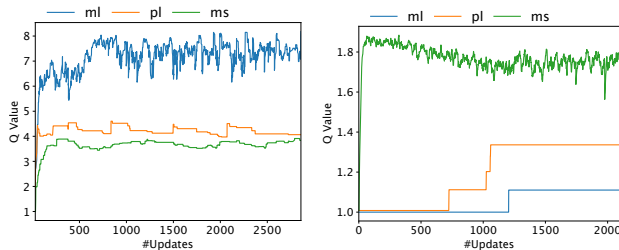


Figure 14: Q values versus number of updates for (a) state index (1, 6, 4, 0) and (b) state index (15, 2, 10, 4).

In Figure 14(a), the Q value for McCore converges rapidly at state index (1, 6, 4, 0), representing low memory access, high port utilization in the last run on L core, and low current bandwidth utilization. Scheduling these workloads on the P core results in approximately 2.5× return. The agent primarily exploits the more frequent *ml* update. On the other hand, Figure 14(b) displays the Q value for state index (15, 2, 10, 4), indicating a memory-intensive situation from the previous run on the S core. In this case, the *ms* action yields a higher return than *ml*, causing the agent to exploit the *ms* action most of the time.

Online vs Offline Learning: Figure 15 compares the performance of the RL agent under various conditions: online only (default), online without optimization, offline only, and a combination of online and offline learning. Performance is normalized to the McCore structure with a random scheduler. In the offline-only scenario,

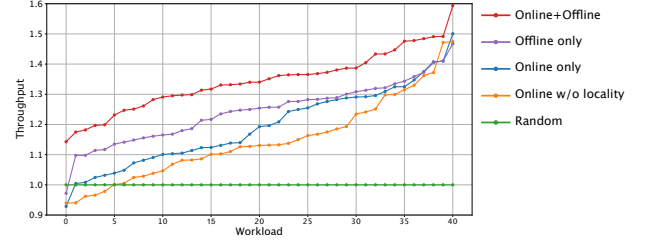


Figure 15: A Comparative Study of Online, Offline, and Locality Sensitivity in RL agent.

the agent employs a QV_table pre-trained from a prior, intricate workload study. The offline-only scheduler does not update the QV_table during scheduling.

Initially, we observe that the RL agent without exploiting locality achieves a 13.7% performance improvement in geomean. On the other hand, the RL agent with optimization techniques attains a 18.6% average performance gain without any pretrained data. The offline-only case, leveraging extensive pretrained knowledge, outperforms the online-only case with a 23.9% average performance gain. However, the RL agent using both pretrained data and adaptive online learning achieves a 34.0% performance improvement, surpassing the offline-only agent by 8.2% on average. This demonstrates the effectiveness of adaptive online learning, even in the presence of pretrained data.

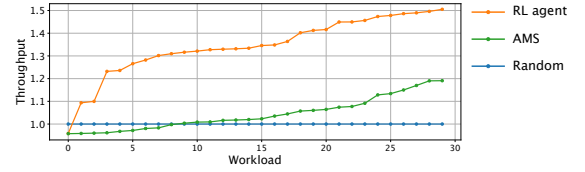


Figure 16: Performance Comparison in a 32-Core System for 30 Workload Mixes.

Scalability of the RL Agent: We examine the scalability of the RL agent in large-scale systems. For this experiment, we set the LLC associativity to 24 for all schemes, and doubled the cache size, bandwidth, and L/S core counts. Figure 16 displays the performance distribution of the RL agent, AMS, and the scaled McCore structure (32 cores) with a random scheduler on 30 mixes of 32 applications. AMS achieves only a 4.1% average improvement due to the drawbacks of way-partitioning in scaling. In contrast, the RL agent (online only) attains a 33.7% average improvement, which is even better than the 16-core case. The reason for this improvement is that doubling the core count in the shared agent design leads to a doubled experience. Consequently, the McCore RL agent scales effectively.

Table 7: Power and Area overhead of McCore RL agent

Structure	Area (mm ²)	Power (mW)
QV table	0.691	23.27
EQ+SQ	0.035	3.11
agent module	0.013	1.17

Hardware Overhead of RL Agent: Table 7 shows the power and area overhead of McCore RL agent. The total power and area of

McCore RL agent is 0.742mm^2 and 27.84mW respectively. Compared to Intel 12900K [43] processor with 125W TDP and 215.25mm^2 die size, McCore RL agent only take 0.34% of area and 0.02% of power. QV_table takes 93% of area and 84% of power in McCore RL agent hardware unit.

8 RELATED WORK

Memory Interference: Memory interference in shared memory [61, 66] and shared cache [56] systems has been extensively studied [33, 59, 81, 90, 91, 97]. Solutions involve partitioning main memory bandwidth [61] and shared caches [12, 33, 34, 59, 78, 80, 81, 103] to mitigate interference. In heterogeneous multicores, small cores can exacerbate memory interference by occupying significant shared LLC and memory bandwidth without providing adequate computation capabilities. McCore addresses this by partitioning shared LLC between L and S cores and conditionally supporting fine-grained memory access for small cores. Though similar to pseudo associative caches [1, 2, 113] in utilizing logical hash-based partitioning and conditional data movement on cache misses, our approach is distinct in its aim and application—primarily focusing on performance isolation in L and S cores. Other approaches include thread scheduling [52, 97] and memory request scheduling [68, 90] to reduce memory interference. Many techniques rely on accurate performance models [52, 91, 92], whereas McCore employs model-free RL-based scheduling and online RL adaptation to handle a variety of situations.

Heterogeneous Multicore: Small cores in heterogeneous multicore were used differently on previous works to accelerate critical section [93], vector execution [96], save energy [7] and increase throughput [43]. McCore uses small core to scale throughput and effectively handles memory-intensive workloads. To better utilize heterogeneous multicore, various scheduling techniques were pursued to improve QoS [75, 114], reliability [70], fairness [101] and energy efficiency [32, 67, 99]. Most of them use the predefined models to schedule heterogeneous multicores aside from PIE [102] and AMS [100]. Energy Aware Scheduling (EAS) [99] that is mostly used for ARM big.LITTLE [7] scheduling, relies on an energy model (EM). On the other hand, McCore use RL-based model-free online learning algorithm to maximize throughput. Intel Thread Director [42] using Enhanced Hardware Feedback Interface (EHFI) [47] to give hints to the OS as to which thread is best to move. It can make a decision more than $10\times$ faster than software scheduler to reach the same decision. We design McCore RL agent as an on-chip module to achieve a similar effect, but we use an adaptive online learning approach rather than taking the predefined policy as in EHFI.

Machine Learning in Computer Architecture: Previously, various machine learning (ML) techniques to improve performance of computer system have been developed including genetic algorithm [11, 38, 54] and perceptron-based approach [15, 53, 98]. Along with them, reinforcement learning (RL) is a promising ML approach for computer architecture. Memory controller [50], prefetcher [16], heterogeneous storage system [86], cache replacement policy [60, 82] and workload management in the cloud [72] can get benefit from RL. Unlike previous works on dynamic resource management (DRM) [20, 31, 55, 62, 63], where they try to adapt mostly offline RL

learning techniques to saving energy in battery-running heterogeneous multicore, McCore uses online RL to maximize performance on high-performance heterogeneous multicore.

9 CONCLUSION

In this paper, we proposed McCore, a novel approach to reorganize the memory hierarchy of heterogeneous multicore systems by utilizing a hardware-based reinforcement learning (RL) scheduler. McCore partitions the large and small core portions in the memory subsystem, employing fine-grained access to alleviate LLC contention and memory bandwidth pressure. We developed a hardware-based scheduler using RL to effectively manage the complexity arising from heterogeneity while leveraging the McCore structure. Our experiments demonstrated the effectiveness of both the McCore structure and the online RL agent. Through holistic management of high-performance heterogeneous multicores, McCore achieved an average performance improvement of 25% compared to the baseline, outperforming other state-of-the-art approaches.

ACKNOWLEDGMENTS

We thank the anonymous reviewers, whose valuable feedback significantly improved the manuscript. This work was supported by Samsung Electronics Co., Ltd (IO201210-07936-01), by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2022-0-00441, Memory-Centric Architecture Using the Reconfigurable PIM Devices), and by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. NRF-2021R1A2B5B01002932). Won Woo Ro is the corresponding author.

REFERENCES

- [1] Anant Agarwal and Stephen D Pudar. 1993. Column-associative caches: A technique for reducing the miss rate of direct-mapped caches. In *Proceedings of the 20th Annual International Symposium on Computer Architecture (ISCA)*. 179–190.
- [2] Anant Agarwal and Steven D Pudar. 1996. Cache memory system and method with multiple hashing functions and hash control storage. US Patent 5,530,958.
- [3] AMD. 2021. AMD EPYC 7763 Processor Product Page. <https://www.amd.com/en/products/cpu/amd-epyc-7763>
- [4] AMD. 2022. AMD Ryzen 9 7950X. <https://www.amd.com/en/products/cpu/amd-ryzen-9-7950x>
- [5] Anandtech. 2021. Golden Cove Microarchitecture (P-Core) Examined. <https://www.anandtech.com/show/16881/a-deep-dive-into-intels-alder-lake-microarchitectures/3>
- [6] Anandtech. 2021. Gracemont Microarchitecture (E-Core) Examined. <https://www.anandtech.com/show/16881/a-deep-dive-into-intels-alder-lake-microarchitectures/4>
- [7] ARM holdings. 2021. ARM big.LITTLE. <https://www.arm.com/technologies/big-little>
- [8] J.-L. Baer and W.-H. Wang. 1988. On the inclusion properties for multi-level cache hierarchies. In *[1988] The 15th Annual International Symposium on Computer Architecture (ISCA). Conference Proceedings*. 73–80.
- [9] Rajeev Balasubramanian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM Trans. Archit. Code Optim. (TACO)* 14, 2, Article 14 (jun 2017), 25 pages.
- [10] Scott Beamer, Krste Asanović, and David Patterson. 2015. The GAP benchmark suite. *arXiv preprint arXiv:1508.03619* (2015).
- [11] Steven J Beatty. 1991. Genetic algorithms and instruction scheduling. In *Proceedings of the 24th Annual International Symposium on Microarchitecture (MICRO)*. 206–211.
- [12] Nathan Beckmann and Daniel Sanchez. 2013. Jigsaw: Scalable software-defined caches. In *Proceedings of the 22nd International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 213–224.
- [13] Richard Bellman. 1957. A Markovian decision process. *Journal of mathematics and mechanics* (1957), 679–684.

- [14] Eli Bendersky. 2018. *Measuring context switching and memory overheads for Linux threads*. <https://eli.thegreenplace.net/2018/measuring-context-switching-and-memory-overheads-for-linux-threads/>
- [15] Rahul Bera, Konstantinos Kanellopoulos, Shankar Balachandran, David Novo, Ataberk Olgun, Mohammad Sadrosadat, and Onur Mutlu. 2022. Hermes: Accelerating Long-Latency Load Requests via Perceptron-Based Off-Chip Load Prediction. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–18.
- [16] Rahul Bera, Konstantinos Kanellopoulos, Anant Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. 2021. Pythia: A customizable hardware prefetching framework using online reinforcement learning. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1121–1137.
- [17] Trevor E. Carlson, Wim Heirman, Stijn Eyerman, Ibrahim Hur, and Lieven Eeckhout. 2014. An Evaluation of High-Level Mechanistic Core Models. *ACM Transactions on Architecture and Code Optimization (TACO)*, Article 5 (2014), 23 pages.
- [18] J Lawrence Carter and Mark N Wegman. 1977. Universal classes of hash functions. In *Proceedings of the Ninth Annual ACM Symposium on Theory of Computing (STOC)*. 106–112.
- [19] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. 2004. R-MAT: A recursive model for graph mining. In *Proceedings of the 2004 SIAM International Conference on Data Mining (SDM)*. SIAM, 442–446.
- [20] Zhuo Chen and Diana Marculescu. 2015. Distributed reinforcement learning for power limited many-core system performance optimization. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1521–1526.
- [21] Chips and Cheese. 2021. *Alder Lake's Caching and Power Efficiency*. <https://chipsandcheese.com/2022/07/07/alder-lakes-caching-and-power-efficiency/>
- [22] Chips and Cheese. 2021. *Gracemont: Revenge of the Atom Cores*. <https://chipsandcheese.com/2021/12/21/gracemont-revenge-of-the-atom-cores/>
- [23] Chips and Cheese. 2021. *Popping the Hood on Golden Cove*. <https://chipsandcheese.com/2021/12/02/popping-the-hood-on-golden-cove/>
- [24] Chisel/FIRRTL. 2021. *Chisel/FIRRTLHardwareCompilerFramework*. <https://www.chisel-lang.org>
- [25] CPU-World. 2023. AMD Opteron 6278 (OS6278WKTGGGU). [https://www.cpu-world.com/CPUs/Bulldozer/AMD-Opteron%206278%20\(OS6278WKTGGGU\).html](https://www.cpu-world.com/CPUs/Bulldozer/AMD-Opteron%206278%20(OS6278WKTGGGU).html) Accessed: 07-07-2023.
- [26] CPU-World. 2023. AMD Third Generation Opteron 8360 SE - OS8360YAL4BGH. <https://www.cpu-world.com/CPUs/K10/AMD-Third%20Generation%20Opteron%208360%20SE%20-%20OS8360YAL4BGH.html> Accessed: 07-07-2023.
- [27] CPU-World. 2023. Intel Xeon E7-8870. <https://www.cpu-world.com/CPUs/Xeon/Intel-Xeon%20E7-8870.html> Accessed: 07-07-2023.
- [28] CPU-World. 2023. Intel Xeon E7-8894 v4. <https://www.cpu-world.com/CPUs/Xeon/Intel-Xeon%20E7-8894%20v4.html> Accessed: 07-07-2023.
- [29] CPU-World. 2023. Intel Xeon MP X7350 - LF80565KH0778M (BX80565X7350). [https://www.cpu-world.com/CPUs/Xeon/Intel-Xeon%20MP%20X7350%20-%20LF80565KH0778M%20\(BX80565X7350\).html](https://www.cpu-world.com/CPUs/Xeon/Intel-Xeon%20MP%20X7350%20-%20LF80565KH0778M%20(BX80565X7350).html) Accessed: 07-07-2023.
- [30] CPU-World. 2023. Intel Xeon MP X7560 - AT80604004869AA (BX80604X7560). [https://www.cpu-world.com/CPUs/Xeon/Intel-Xeon%20MP%20X7560%20-%20AT80604004869AA%20\(BX80604X7560\).html](https://www.cpu-world.com/CPUs/Xeon/Intel-Xeon%20MP%20X7560%20-%20AT80604004869AA%20(BX80604X7560).html) Accessed: 07-07-2023.
- [31] Aryan Deshwal, Syrine Belakaria, Ganapati Bhat, Janardhan Rao Doppa, and Partha Pratim Pande. 2021. Learning pareto-frontier resource management policies for heterogeneous socs: An information-theoretic approach. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 607–612.
- [32] Bryan Donyanavard, Tiago Mück, Santanu Sarma, and Nikil Dutt. 2016. SPARTA: Runtime task allocation for energy efficient heterogeneous manycores. In *2016 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ ISSS)*. IEEE, 1–10.
- [33] David Eklov, Nikos Nikolieris, David Black-Schaffer, and Erik Hagersten. 2011. Cache pirating: Measuring the curse of the shared cache. In *2011 International Conference on Parallel Processing (ICPP)*. IEEE, 165–175.
- [34] Nosayba El-Sayed, Anurag Mukkara, Po-An Tsai, Harshad Kasture, Xiaosong Ma, and Daniel Sanchez. 2018. KPart: A hybrid cache partitioning-sharing technique for commodity multicores. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 104–117.
- [35] Hadi Esmaeilzadeh, Emily Blem, Renee St. Amant, Karthikeyan Sankaralingam, and Doug Burger. 2011. Dark silicon and the end of multicore scaling. In *Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA)*. 365–376.
- [36] Alexandra Fedorova, David Vengerov, and Daniel Doucette. 2007. Operating system scheduling on heterogeneous core systems. In *Proceedings of the Workshop on Operating System Support for Heterogeneous Multicore Architectures*.
- [37] Wim Heirman, Stijn Eyerman, Kristof Du Bois, and Ibrahim Hur. 2021. Automatic sublining for efficient sparse memory accesses. *ACM Transactions on Architecture and Code Optimization (TACO)* 18, 3 (2021), 1–23.
- [38] Chien-Chin Huang, Gu Jin, and Jinyang Li. 2020. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1341–1355.
- [39] Intel. 2021. *Intel Xeon Platinum 8380 Processor (60M Cache, 2.30 GHz) Product Specifications*. <https://www.intel.com/content/www/us/en/products/sku/212287/intel-xeon-platinum-8380-processor-60m-cache-2-30-ghz/specifications.html>
- [40] Intel. 2021. *Introduction to Cache Allocation Technology*. Intel Developer Zone. <https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-cache-allocation-technology.html> Accessed: 2023-07-07.
- [41] Intel Corporation. 2021. *Blueprint Series: 12th Gen Intel® Core™ Processors*. <https://download.intel.com/newsroom/2021/manufacturing/12th-Gen-Blueprint-Series-Presentation.pdf>
- [42] Intel Corporation. 2021. *Intel Architecture Day 2021*. <https://www.intel.com/content/www/us/en/newsroom/resources/press-kit-architecture-day-2021.html>
- [43] Intel Corporation. 2021. *Intel® Core™ i9-12900K Processor*. <https://www.intel.com/content/www/us/en/products/sku/134599/intel-core-i912900k-processor-30m-cache-up-to-5-20-ghz/specifications.html>
- [44] Intel Corporation. 2021. *Pin - A Dynamic Binary Instrumentation Tool*. <https://www.intel.com/content/www/us/en/developer/articles/tool/pin-a-dynamic-binary-instrumentation-tool.html>
- [45] Intel Corporation. 2022. *12th Generation Intel® Core™ Processors Datasheet, Volume 1 of 2*. Intel Corporation. <https://edc.intel.com/content/www/us/en/design/ipa/software-development-platforms/client/platforms/alder-lake-desktop/12th-generation-intel-core-processors-datasheet-volume-1-of-2/001/operating-systems-support/> Rev. 007.
- [46] Intel Corporation. 2022. *Intel® 64 and IA-32 Architectures Optimization Reference Manual*. Intel Corporation. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html> Version. 045.
- [47] Intel Corporation. 2022. *Intel® Architecture Instruction Set Extensions and Future Features*. Intel Corporation. <https://software.intel.com/content/dam/develop/external/us/en/documents-tps/architecture-instruction-set-extensions-programming-reference.pdf> Version. 046.
- [48] Intel Corporation. 2022. *Intel® Memory Latency Checker (Intel® MLC)*. <https://www.intel.com/content/www/us/en/download/736633/763324/intel-memory-latency-checker-intel-mlc.html>
- [49] Intel Corporation. 2022. *lmbench*. <https://github.com/intel/lmbench>
- [50] Engin Ipek, Onur Mutlu, José F. Martínez, and Rich Caruana. 2008. Self-Optimizing Memory Controllers: A Reinforcement Learning Approach. In *2008 International Symposium on Computer Architecture (ISCA)*. 39–50.
- [51] Aamer Jaleel, Kevin B. Theobald, Simon C. Steely, and Joel Emer. 2010. High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP). In *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA '10)*. Association for Computing Machinery, New York, NY, USA, 60–71.
- [52] Xiaowei Jiang, Asit Mishra, Li Zhao, Ravishankar Iyer, Zhen Fang, Sadagopan Srinivasan, Srihari Makineni, Paul Brett, and Chita R Das. 2011. ACCESS: Smart scheduling for asymmetric cache CMPs. In *2011 IEEE 17th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 527–538.
- [53] Daniel A Jiménez and Calvin Lin. 2001. Dynamic branch prediction with perceptrons. In *Proceedings of the Seventh International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 197–206.
- [54] Ajay M Joshi, Lieven Eeckhout, Lizy K John, and Ciji Isen. 2008. Automated microprocessor stressmark generation. In *2008 IEEE 14th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 229–239.
- [55] Ryan Gary Kim, Wonje Choi, Zhuo Chen, Janardhan Rao Doppa, Partha Pratim Pande, Diana Marculescu, and Radu Marculescu. 2017. Imitation learning for dynamic VFI control in large-scale manycore systems. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 25, 9 (2017), 2458–2471.
- [56] Seongbeom Kim, Dhruba Chandra, and Yan Solihin. 2004. Fair cache sharing and partitioning in a chip multiprocessor architecture. In *Proceedings. 13th International Conference on Parallel Architecture and Compilation Techniques (PACT)*. IEEE, 111–122.
- [57] Daniel Lemire. 2022. *Testing memory-level parallelism*. <https://github.com/lemire/testingmlp>
- [58] Sheng Li, Jung Ho Ahn, Richard D Strong, Jay B Brockman, Dean M Tullsen, and Norman P Jouppi. 2009. McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures. In *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 469–480.
- [59] Jiang Lin, Qingda Lu, Xiaoning Ding, Zhao Zhang, Xiaodong Zhang, and Ponnuwamy Sadayappan. 2008. Gaining insights into multicore cache partitioning: Bridging the gap between simulation and real systems. In *2008 IEEE 14th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 367–378.
- [60] Evan Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. 2020. An imitation learning approach for cache replacement. In *International Conference on Machine Learning (ICML)*. PMLR, 6237–6247.

- [61] Fang Liu, Xiaowei Jiang, and Yan Solihin. 2010. Understanding how off-chip memory bandwidth partitioning in Chip Multiprocessors affects system performance. In *HPCA - 16 2010 The Sixteenth International Symposium on High-Performance Computer Architecture (HPCA)*. 1–12.
- [62] Sumit K Mandal, Ganapati Bhat, Janardhan Rao Doppa, Partha Pratim Pande, and Umit Y Ogras. 2020. An energy-aware online learning framework for resource management in heterogeneous platforms. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 25, 3 (2020), 1–26.
- [63] Sumit K Mandal, Ganapati Bhat, Chetan Arvind Patil, Janardhan Rao Doppa, Partha Pratim Pande, and Umit Y Ogras. 2019. Dynamic resource management of heterogeneous mobile platforms via imitation learning. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27, 12 (2019), 2842–2854.
- [64] Larry W McVoy, Carl Staelin, et al. 1996. Imbench: Portable Tools for Performance Analysis. In *USENIX Annual Technical Conference (ATC)*. San Diego, CA, USA, 279–294.
- [65] Micron Technology, Inc. 2020. *DDR5: Key Module Features*. Technical Report. Micron Technology, Inc. https://media-www.micron.com/-/media/client/global/documents/products/technical-marketing-brief/ddr5_key_module_features_tech_brief.pdf?la=zh-tw&rev=f3ca96bed7d9427ba72b4c192dfac56
- [66] Thomas Moscibroda and Onur Mutlu. 2007. Memory Performance Attacks: Denial of Memory Service in Multi-Core Systems. In *16th USENIX Security Symposium (USENIX Security 07)*. USENIX Association, Boston, MA. <https://www.usenix.org/conference/16th-usenix-security-symposium/memory-performance-attacks-denial-memory-service-multi>
- [67] Thannirmalai Somu Muthukaruppan, Mihai Pricopi, Vanchinathan Venkataramani, Tulika Mitra, and Sanjay Vishin. 2013. Hierarchical power management for asymmetric multi-core in dark silicon era. In *2013 50th ACM/EDAC/IEEE Design Automation Conference (DAC)*. IEEE, 1–9.
- [68] Onur Mutlu and Thomas Moscibroda. 2008. Parallelism-aware batch scheduling: Enhancing both performance and fairness of shared DRAM systems. In *2008 International Symposium on Computer Architecture (ISCA)*. IEEE, 63–74.
- [69] Samuel Naffziger, Noah Beck, Thomas Burd, Kevin Lepak, Gabriel H Loh, Mahesh Subramony, and Sean White. 2021. Pioneering chiplet technology and design for the amd epyc™ and ryzen™ processor families: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 57–70.
- [70] Ajeya Naithani, Stijn Eyerman, and Lieven Eeckhout. 2017. Reliability-aware scheduling on heterogeneous multicore processors. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 397–408.
- [71] NC State University. 2008. *FreePDK45TM*. <https://eda.ncsu.edu/freepdk/freepdk45/>
- [72] Rajiv Nishtala, Vinicius Petrucci, Paul Carpenter, and Magnus Sjalander. 2020. Twig: Multi-Agent Task Management for Colocated Latency-Critical Cloud Services. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 167–179.
- [73] Geraldo F Oliveira, Juan Gómez-Luna, Lois Orosa, Saugata Ghose, Nandita Vijaykumar, Ivan Fernandez, Mohammad Sadrosadati, and Onur Mutlu. 2021. DAMOV: A new methodology and benchmark suite for evaluating data movement bottlenecks. *IEEE Access* 9 (2021), 134457–134502.
- [74] Chandandeep Singh Pabla. 2009. Completely fair scheduler. *Linux Journal* 2009, 184 (2009), 4.
- [75] Vinicius Petrucci, Michael A Laurenzano, John Doherty, Yunqi Zhang, Daniel Mosse, Jason Mars, and Lingjia Tang. 2015. Octopus-man: Qos-driven task management for heterogeneous multicores in warehouse-scale computers. In *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 246–258.
- [76] Princeton University. 2007. *The PARSEC Benchmark Suite*. <https://parsec.cs.princeton.edu>
- [77] Moinuddin K Qureshi and Gabe H Loh. 2012. Fundamental latency trade-off in architecting dram caches: Outperforming impractical sram-tags with a simple and practical design. In *2012 45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 235–246.
- [78] Moinuddin K Qureshi and Yale N Patt. 2006. Utility-based cache partitioning: A low-overhead, high-performance, runtime mechanism to partition shared caches. In *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'06)*. IEEE, 423–432.
- [79] Daniel Sanchez and Christos Kozyrakis. 2010. The ZCache: Decoupling ways and associativity. In *2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 187–198.
- [80] Daniel Sanchez and Christos Kozyrakis. 2011. Vantage: Scalable and efficient fine-grain cache partitioning. In *Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA)*. 57–68.
- [81] Andreas Sandberg, Andreas Sembrant, Erik Hagersten, and David Black-Schaffer. 2013. Modeling performance variation due to cache sharing. In *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 155–166.
- [82] Subhash Sethumurugan, Jieming Yin, and John Sartori. 2021. Designing a cost-effective cache replacement policy using machine learning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 291–303.
- [83] André Seznec. 1993. A case for two-way skewed-associative caches. In *Proceedings of the 20th Annual International Symposium on Computer Architecture (ISCA)*. IEEE Computer Society, 169–178.
- [84] Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. 2002. Automatically characterizing large scale program behavior. In *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 45–57.
- [85] Jinuk Luke Shin, Kenway Tam, Dawei Huang, Bruce Petrick, Ha Pham, Changku Hwang, Hongping Li, Alan Smith, Timothy Johnson, Francis Schumacher, David Greenhill, Ana Sonia Leon, and Allan Strong. 2010. A 40nm 16-core 128-thread CMT SPARC SoC processor. In *2010 IEEE International Solid-State Circuits Conference - (ISSCC)*. 98–99.
- [86] Gagandeep Singh, Rakesh Nadig, Jisung Park, Rahul Bera, Nastaran Hajinazar, David Novo, Juan Gómez-Luna, Sander Stuijk, Henk Corporaal, and Onur Mutlu. 2022. Sibyl: Adaptive and Extensible Data Placement in Hybrid Storage Systems Using Online Reinforcement Learning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)* (New York, New York) (ISCA '22). ACM, New York, NY, USA, 320–336.
- [87] Livio Soares and Michael Stumm. 2010. FlexSC: Flexible System Call Scheduling with Exception-Less System Calls. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [88] Mathias Spjuth, Martin Karlsson, and Erik Hagersten. 2005. Skewed caches from a low-power perspective. In *Proceedings of the 2nd conference on Computing frontiers*. 152–160.
- [89] Standard Performance Evaluation Corporation. 2021. *SPEC CPU® 2017*. <https://www.spec.org/cpu2017>
- [90] Lavanya Subramanian, Donghyuk Lee, Vivek Seshadri, Harsha Rastogi, and Onur Mutlu. 2016. BLISS: Balancing Performance, Fairness and Complexity in Memory Access Scheduling. *IEEE Transactions on Parallel and Distributed Systems (TPDS)* 27, 10 (2016), 3071–3087.
- [91] Lavanya Subramanian, Vivek Seshadri, Arnab Ghosh, Samira Khan, and Onur Mutlu. 2015. The application slowdown model: Quantifying and controlling the impact of inter-application interference at shared caches and main memory. In *2015 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 62–75.
- [92] Lavanya Subramanian, Vivek Seshadri, Yoongu Kim, Ben Jaiyen, and Onur Mutlu. 2013. MISE: Providing performance predictability and improving fairness in shared main memory systems. In *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 639–650.
- [93] M. Aater Suleman, Onur Mutlu, Moinuddin K. Qureshi, and Yale N. Patt. 2009. Accelerating Critical Section Execution with Asymmetric Multi-Core Architectures. In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 253–264.
- [94] Richard S Sutton and Andrew G Barto. 2018. *Reinforcement learning: An introduction*. MIT press.
- [95] Synopsys. 2021. *Synopsys Design Compiler®*. <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html>
- [96] Tuan Ta, Khalid Al-Hawaj, Nick Cebery, Yanghui Ou, Eric Hall, Courtney Golden, and Christopher Batten. 2022. big.VLITTLE: On-Demand Data-Parallel Acceleration for Mobile Systems on Chip. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 181–198.
- [97] Lingjia Tang, Jason Mars, Neil Vachharajani, Robert Hundt, and Mary Lou Soffa. 2011. The impact of memory subsystem resource sharing on datacenter applications. In *2011 38th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 283–294.
- [98] Elvira Teran, Zhe Wang, and Daniel A Jiménez. 2016. Perceptron learning for reuse prediction. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–12.
- [99] The kernel development community. 2022. *The Linux Kernel 6.1.0-rc5*. <https://www.kernel.org/doc/html/latest/scheduler/sched-energy.html>
- [100] Po-An Tsai, Changping Chen, and Daniel Sanchez. 2018. Adaptive scheduling for systems with asymmetric memory hierarchies. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 641–654.
- [101] Kenzo Van Craeynest, Shoaib Akram, Wim Heirman, Aamer Jaleel, and Lieven Eeckhout. 2013. Fairness-aware scheduling on single-ISA heterogeneous multicores. In *Proceedings of the 22nd International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 177–187.
- [102] Kenzo Van Craeynest, Aamer Jaleel, Lieven Eeckhout, Paolo Narvaez, and Joel Emer. 2012. Scheduling heterogeneous multi-cores through performance impact estimation (PIE). In *2012 39th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 213–224.
- [103] Xiaodong Wang, Shuang Chen, Jeff Setter, and José F Martínez. 2017. SWAP: Effective fine-grain management of shared last-level caches with minimum hardware support. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 121–132.

- [104] Christopher JCH Watkins and Peter Dayan. 1992. Q-learning. *Machine learning* 8, 3 (1992), 279–292.
- [105] Zhipeng Wei, Zehan Cui, and Mingyu Chen. 2015. Cracking Intel Sandy Bridge's Cache Hash Function. *arXiv preprint arXiv:1508.03767* (2015).
- [106] WikiChip. 2021. *Intel Ice Lake (server) - Microarchitectures - WikiChip*. [https://en.wikichip.org/wiki/intel/microarchitectures/ice_lake_\(server\)](https://en.wikichip.org/wiki/intel/microarchitectures/ice_lake_(server))
- [107] WikiChip. 2021. *Skylake (client) - Microarchitectures - Intel*. [https://en.wikichip.org/wiki/intel/microarchitectures/skylake_\(client\)](https://en.wikichip.org/wiki/intel/microarchitectures/skylake_(client))
- [108] WikiChip. 2021. *Zen 4 - Microarchitectures - AMD*. https://en.wikichip.org/wiki/amd/microarchitectures/zen_4
- [109] WikiChips. 2021. *Alder Lake Die:ADL-S*. https://en.wikichip.org/wiki/intel/microarchitectures/alder_lake#ADL-S_288P.2B8E.29
- [110] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM (CACM)* 52, 4 (2009), 65–76.
- [111] Doe Hyun Yoon, Min Kyu Jeong, and Mattan Erez. 2011. Adaptive granularity memory systems: A tradeoff between storage efficiency and throughput. In *Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA)*. 295–306.
- [112] Doe Hyun Yoon, Min Kyu Jeong, Michael Sullivan, and Mattan Erez. 2012. The dynamic granularity memory system. In *2012 39th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 548–560.
- [113] Chenxi Zhang, Xiaodong Zhang, and Yong Yan. 1997. Multi-column implementations for cache associativity. In *Proceedings International Conference on Computer Design VLSI in Computers and Processors (ICCD)*. 504–509.
- [114] Yuhao Zhu, Matthew Halpern, and Vijay Janapa Reddi. 2015. Event-based scheduling for energy-efficient qos (eqos) in mobile web applications. In *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 137–149.